

Heretto CCMS API Reference Guide





Contents

REST API	4
Root Collections	4
HTTP Request Headers	5
Generate a Token	6
File Access	8
Documents	8
Get Document Information	8
Create a Document	12
Get Document Contents	16
Update Document Contents	19
Update Metadata Method	22
Get Document Bundle	24
Get Published Document	26
Retrieve Resource Dependencies	27
Retrieve Resource Inbound Links	29
Folders	31
Retrieve Folder	31
Create Folder Method	36
Locks	39
Retrieve Lock	39
Add lock	41
Remove Lock	43
Branches and Releases	44
Retrieve Branches	44
Retrieve Releases	46
Search	51
Webhooks	68
Triggering Actions on External Servers with Webhooks (Notify)	70
Pulling Content from Heretto CCMS to External Servers (NotifyPull)	71



- Publishing Content on External Servers with Webhooks (NotifyPull)..... 72
- Implement a Webhook Integration 73
 - Webhook Authentication and Security 74
 - Webhook Configuration Files 76
 - Webhook Notification Request 80
 - Webhook Status Request 83
- Configure Webhook Timeout 86
- Send a Webhook Notification on Publish..... 88
- Send a Webhook Notification on Demand 91



Document History

Version	Updates	Date
A	Initial document version.	July 7, 2021
B	Updated Retrieve Releases on page 46.	August 17, 2021
C	Updated curl request structures	January 27, 2022
D	Added an example to the Create a Document on page 12 method.	May 3, 2024
D	Non-Disclosure Agreement (NDA) is no longer required for customers to see this document.	November 5, 2024
E	▪ Removed the <code>resolution</code> parameter from the Get Document Bundle on page 24 method. ▪ Updated the description of the Create a Document on page 12 method. ▪ Removed an unnecessary ampersand (@) from both request examples in the Update Document Contents on page 19 method.	May 7, 2025
F	Updated Webhook Configuration Files on page 76 for clarity.	September 18, 2025
G	Updated Update Document Contents on page 19 to include a note about the required document type (doctype) declaration.	December 8, 2025



REST API

Heretto REST API provides your organization with general file access for updating content or data associated with content.

☆ **IMPORTANT:** Our realm is “exist”.

Capabilities

- Access your files via the Uniform Resource Identifier (URI) or Universally Unique Identifier (UUID)
- Search your files based on content and metadata
- Inspect links
- Upload and update files

Root Collections

List of the standard root REST Collections.

Collection	Description
db	Direct access to any resource in the database for modification by users with the appropriate privileges. Typically this is disabled for all non-database administrator accounts.
repositories	Access to documents by repository.
localization-jobs	Access to all localization jobs.



HTTP Request Headers

Field	Description	Allowed Values
Accept	Used to determine the response type from the request. REST Resources which are complete and in compliance will always allow <code>application/xml</code> , some will also allow <code>text/vnd.jorsek.uuid</code> . The default response may vary on a case by case basis, but typically it's <code>application/xml</code> for GET and <code>text/vnd.jorsek.uuid</code> for PUT and POST.	<code>text/</code> <code>vnd.jorsek.uuid</code> <code>application/xml</code>



Generate a Token

You can generate a token to authenticate API requests instead of using your Heretto CCMS password. Use this token with any tool or application that makes API calls (such as cURL, WebDAV, Postman, or custom scripts).

Create separate tokens for different applications or scripts. This enables you to revoke a token without affecting other integrations.

Context

When single sign-on (SSO) is configured in the CCMS, you have to generate a token. Otherwise, you can use your CCMS password. However, for security reasons, using a token is strongly recommended regardless of whether SSO is configured.

Procedure

- 1 Sign in to Heretto CCMS.
- 2 In a web browser, navigate to `https://{organizationId}/tools/token-management/tokens.xql`
Where `{organizationId}` is the subdomain you use to access Heretto CCMS. For example, if you access Heretto CCMS at `https://thunderbird.heretto.com`, use `thunderbird` as the `organizationId`.
Example: `https://thunderbird.heretto.com/tools/token-management/tokens.xql`
- 3 In the Token name field, add a meaningful name for the token.
The name will be visible in the tokens interface.
Example: `Oxygen Webdav Token`, or `CI/CD Pipeline`, or `Publishing Script`
- 4 Click **create token**.
- 5 Copy the login and password (token) information and keep it in a secure place.

☆ **IMPORTANT:** Once you navigate away from the page with your login and token, you won't be able to see the token again.

Result



Your token is now active. It will remain active until you manually invalidate it.



File Access

Documents

Get Document Information

You use the GET method to retrieve document information.

Overview

Type	Value
HTTP Method	GET
URI (via all-files)	/rest/all-files/{file-or-folder-id}
Returns	A representation of the document

Request Headers

Type	Value	Description
If-Modified-Since	The HTTP-date to compare against	Optional - If the resource has not been modified since the supplied date, a 304 (not modified) will be returned with no content.



Response Headers

Type	Value
Content-Type	The content type of the document (For example: <code>application/xml</code>)

Parameters

No additional parameters are supported.

Example

Request:

```
curl -i -u {your.user@org.com:{token}} https://{your-org}.heretto.com/rest/all-files/  
ce2a26b0-1d34-11e2-92fc-485d6037d75d
```

Response (the document content):

```
HTTP/1.1 200 OK  
Content-Type: application/xml; charset=UTF-8  
  
<resource id="ce2a26b0-1d34-11e2-92fc-485d6037d75d" created="1358781244191" last-modified="1358781244191">  
  <name>topic.dita</name>  
  <owner>the.user@user.com</owner>  
  <group>group2</group>  
  <mode>504</mode>  
  <mime-type>application/xml</mime-type>  
  <metadata>
```



```
        <meta name="status">Draft</meta>
    </metadata>
</resource>
```

Schema

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema" elementFormDefault="qualified">
  <xs:element name="resource">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="name" type="xs:string"/>
        <xs:element name="owner" type="xs:string"/>
        <xs:element name="group" type="xs:string"/>
        <xs:element name="mode" type="xs:integer"/>
        <xs:element name="mime-type" type="xs:string"/>
        <xs:element ref="metadata"/>
      </xs:sequence>
      <xs:attribute name="created" use="required" type="xs:integer"/>
      <xs:attribute name="id" use="required"/>
      <xs:attribute name="last-modified" use="required" type="xs:integer"/>
      <xs:attribute name="locked-by" use="optional" type="xs:string"/>
    </xs:complexType>
  </xs:element>

  <xs:element name="metadata">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="meta" minOccurs="0">
          <xs:complexType>
            <xs:simpleContent>
```



```
        <xs:extension base="xs:string">
            <xs:attribute name="name" use="required"/>
        </xs:extension>
    </xs:simpleContent>
</xs:complexType>
</xs:element>
</xs:sequence>
</xs:complexType>
</xs:element>
</xs:schema>
```



Create a Document

You use the POST method on a folder to create or update a document representation (or placeholder). Once the representation of the document is created, to add content to it, you need to parse the response for the newly created document `resourceID`, then call the *Update Document Contents* method to put any content there.

Overview

Type	Value
HTTP Method	POST
URI	<code>/rest/all-files/{folder-id}</code>
Returns	Empty, <code>201 CREATED</code> if created, or <code>200 OK</code> if updated

Request Headers

Type	Value	Description
Content-Type	A valid content type to store the document as	Required - Must be set to <code>application/xml</code>
Content-Length	The length/size of the content to be stored	Optional, but recommended - If not supplied the system will attempt to determine the content length



POST body

The POST body takes the following format:

```
<resource>
  <name>{resource-file-name-with-extension}</name>
  <mime-type>{mime-type}</mime-type>
  <metadata>
    <meta name="{metadata-key-name}">{metadata-value}</meta>
    <meta name="content-type">{DOCTYPE}</meta>
  </metadata>
</resource>
```

The `<name>`, `<mime-type>`, and `<content-type>` fields are required.

The `<mime-type>` for DITA XML should be `application/xml`

Additional optional fields include:

<code><owner></code>	The username of the user that will be assigned as the file's owner
<code><group></code>	The group that will be assigned to this file
<code><mode></code>	The permissions mode for this file (Unix format)

Response Headers

No additional headers are returned.



Parameters

No additional parameters are supported.

Example: Creating a new XML document

Request (creating a new XML file in repository root):

```
curl -i -H "Content-Type: application/xml" -X POST -u {your.user@org.com:{token}} -d
"<resource><name>topic.dita</name><mime-type>application/xml</mime-type></resource>" https://{your-
org}.heretto.com/rest/all-files/ce2a26b0-1d34-11e2-92fc-485d6037d75d
```

Response:

```
HTTP/1.1 201 CREATED

<resource id="ce2a26b0-1d34-11e2-92fc-485d6037d75d">
  <name>topic.dita</name>
  <owner>theuser@user.com</owner>
  <group>group2</group>
  <mode>504</mode>
  <metadata/>
</resource>
```

Example: Creating (uploading) an image file

Request (uploading a local image file into the repository):



```
curl -i -H "Content-Type: image/jpeg" -X POST -u {your.user@horg.com:{token}} -d
"<resource><name>{image_name.jpg}</name><mime-type>image/jpeg</mime-type></resource>" https://{your-
org}.heretto.com/rest/all-files/d0dd21f0-ed25-11ee-88b8-0242039bae5e
```

Response:

HTTP/2 201

```
<resource id="a470ef20-04ae-11ef-bfec-0242039bae5e" created="1714233360146" last-modified="1714233360146"
folder-uuid="d0dd21f0-ed25-11ee-88b8-0242039bae5e">
  <name>{IMG_0106.jpg}</name>
  <xml-db-uri>{file-path.png}</xml-db-uri>
  <owner>{your.user@org.com}</owner>
  <group>{__org-name}</group>
  <mode>504</mode>
  <mime-type>image/png</mime-type>
  <metadata/>
</resource>
```



Get Document Contents

You use the GET method to retrieve a document content or data.

Overview

Type	Value
HTTP Method	GET
URI (via all-files)	/rest/all-files/{document-id}/content
Returns	The file content/data

Request Headers

Type	Value	Description
If-Modified-Since	The HTTP-date to compare against	Optional - If the resource has not been modified since the supplied date, a 304 (not modified) will be returned with no content



Response Headers

Type	Value
Content-Type	The content type of the document (For example: <code>application/xml</code> or <code>image/jpeg</code> , etc.)

Parameters

Name	Value	Description
view	The UUID of a context map	Specifies the display of the content. This is the XSLT applied to the content. <code>web</code> is the view used for previews in Heretto CCMS. These views can also be custom.
map-context	The UUID of a context map	This map is used to resolve keys, etc.
filter	The UUID of a ditaval file or a dynamic ditaval	This ditaval is applied to the topic. In the case of a dynamic ditaval, the <code><val></code> element can be supplied as the value. This value should be URL encoded.
link-processor	Empty or <code>content</code>	If the <code>content</code> is not specified, links are corrected to the Share URLs for files. If the <code>content</code> is specified, links are corrected to the content endpoint. The current view, map-context, and filter are kept.

Example

Request:



```
curl -i -u {your.user@org.com:{token}} https://{your-org}.heretto.com/rest/repositories/test/documents/ce2a26b0-1d34-11e2-92fc-485d6037d75d/content
```

Response (the document's content):

```
HTTP/1.1 200 OK
Content-Type: application/xml; charset=UTF-8

<!DOCTYPE task PUBLIC "-//OASIS//DTD DITA General Task//EN" "http://docs.oasis-open.org/dita/v1.1/OS/dtd/task.dtd">
<task id="taskId">
  <title>Task title</title>
  <shortdesc/>
  <taskbody>
    <context>
      <p>Context for the current task</p>
    </context>
    <steps>
      <step>
        <cmd>Task step.</cmd>
      </step>
    </steps>
  </taskbody>
</task>
```



Update Document Contents

You use the PUT method to add or update the content in an existing document.

☆ **IMPORTANT:** DITA files require a document type (doctype) declaration at the top of the file that corresponds to its information type. For example, a task topic would have `<!DOCTYPE task PUBLIC "-//OASIS//DTD DITA Task//EN" "task.dtd">` right after the `<?xml?>` declaration. If a file is missing the doctype, the system shows a validation error.

Overview

Type	Value
HTTP Method	PUT
URI	<code>/rest/all-files/{document-id}/content</code>
Returns	Empty, <code>201 CREATED</code> if created, or <code>200 OK</code> if updated

Request Headers

Type	Value	Description
Content-Type	A valid content type to store the document as	Required - The document Mime-Type will be set using this value



Type	Value	Description
Content-Length	The length/size of the content to be stored	Optional but recommended - If not supplied the system will attempt to determine the content length

Response Headers

No additional headers are returned.

Parameters

No additional parameters are supported.

Example: Storing XML content

Request (uploading a new XML document):

```
curl -i -H "Content-Type: application/xml" 2222-X PUT -d topic.dita {your.user@org.com:{token}} https://{your-org}.heretto.com/rest/all-files/8f3ed200-cbba-11e1-ac51-c82a1446d15c/content
```

Response:

```
HTTP/1.1 200 OK
```

Example: Storing image content

Request (uploading a new image):



```
curl -i -H "Content-Type:" -X PUT --data-binary test.jpg {your.user@org.com:{token}} https://{your-org}.heretto.com/rest/all-files/8f3ed200-cbba-11e1-ac51-c82a1446d15c/content
```

☆ **IMPORTANT:** When uploading binary files, ensure that the Content-Type is either well-defined, or empty, and that the data is correctly set as binary (when using `curl`, the `--data-binary` flag).

Response:

```
HTTP/1.1 201 CREATED
```



Update Metadata Method

You use the PUT method to update metadata values in the repository.

Overview

Type	Value
HTTP Method	PUT
URI	/rest/all-files/{document-id}/metadata
Returns	Empty or 200 OK if successful

Request Headers

No request headers are required.

PUT body

The PUT body takes the following format:

```
<metadata>
  <meta name="{field-name}">{new-field-value}</meta>
  <meta name="{field-name-2}">{new-field-value-2}</meta>
  ...
</metadata>
```

You may supply as many `<meta />` elements as you like to update multiple fields.



Response Headers

No additional headers are returned.

Parameters

No additional parameters are supported.

Example: Updating the document metadata

Request:

```
curl -i -H "Content-Type: application/xml" -u {your.user@org.com:{token}} -X PUT -d "<metadata><meta
name='test'>new-value</meta></metadata>" https://{your-org}.heretto.com/rest/all-files/
75927d70-9e4a-11e3-9b84-e4ce8f3a81be/metadata
```

Response:

```
HTTP/1.1 200 OK
Date: Sun, 08 Jun 2014 16:30:33 GMT
Cache-Control: no-store, no-cache, must-revalidate
Cache-Control: post-check=0, pre-check=0
Pragma: no-cache
Set-Cookie: JSESSIONID=10rrpmym2mjtitfwfn1l8yu35;Path=/
Expires: Thu, 01 Jan 1970 00:00:00 GMT
Content-Length: 0
Server: Jetty(8.1.11.v20130520)
```



Get Document Bundle

You use the GET method to retrieve a document bundle, which is a ZIP file containing the target document and all of its dependencies.

Overview

Type	Value
HTTP Method	GET
URI	/rest/all-files/{document-uuid}/bundle
Returns	The root file and file dependencies in an application/zip stream

Parameters

Name	Values	Description
filter	Optional: A UUID of a DITaval filter.	If specified, the <ditavalref> elements are ignored.
locale	Optional: A locale name.	For each file downloaded, the latest localization for the file is pulled. If no localized version of the file is available, the file in the repository is used.



Request Example

```
curl -i -u {your.user@org.com:{token}} https://{your-org}.heretto.com/rest/all-files/  
ce2a26b0-1d34-11e2-92fc-485d6037d75d/bundle?locale=ja
```

Manifest

The `__manifest__.xml` file is written in the root of the bundle. The manifest identifies the root file (usually a DITA map), and all the other dependency files, with corresponding metadata.



Get Published Document

You use the GET method to retrieve a publish output bundle, which is a ZIP file containing the published output for a document.

Overview

HTTP Method

GET

URI

/rest/all-files/{publish-output-identifier}/download

Returns

The zipped output of the publish

Depending on your Heretto CCMS version or configuration, the {publish-output-identifier} may either be the:

- UUID of a given publishing job (for example: `ce2a26b0-1d34-11e2-92fc-485d6037d75d`)
- URI of a given publishing job output (for example: `https://{your-organization}.heretto.com/rest/job/pub-012324567/jmc_publish_map_a.zip`)

Parameters

No additional parameters are supported.

Request Example

```
curl -i -u {your.user@org.com:{token}} https://{your-org}.heretto.com/rest/all-files/ce2a26b0-1d34-11e2-92fc-485d6037d75d/download
```



Retrieve Resource Dependencies

You use the GET method to retrieve all of the dependencies for a resource.

Overview

Type	Value
HTTP Method	GET
URI	/rest/all-files/{resource-id}/links/dependencies
Returns	An XML fragment describing the dependencies for this resource

Request Headers

Type	Value	Description
Accept	application/xml	Optional (default)

Response Headers

Type	Value
Content-Type	application/xml; charset=UTF-8



Parameters

No additional parameters are supported.

Example

Request:

```
curl -i -H "Accept: application/xml" -u {your.user@org.com:{token}} https://{your-org}.heretto.com/rest/all-files/8f411bf0-cbba-11e1-ac51-c82a1446d15c/links/dependencies
```

Response:

```
HTTP/1.1 200 OK
```

```
<?xml version="1.0" ?>
<dependencies target-resource-id="7ab92790-9e4a-11e3-9b84-e4ce8f3a81be"
  target-resource-uri="product-documentation/documents/Demo_2/map.ditamap">
  <resource id="7ab5f340-9e4a-11e3-9b84-e4ce8f3a81be" relative-uri="doc-C.dita"/>
  <resource id="7b03a8a0-9e4b-11e3-9b84-e4ce8f3a81be"
    relative-uri="super_bike_motorcycle_photo-779085.jpg"/>
  <resource id="7ab44590-9e4a-11e3-9b84-e4ce8f3a81be" relative-uri="airplane_overview.dita"/>
  <resource id="7ab026e0-9e4a-11e3-9b84-e4ce8f3a81be" relative-uri="A_task.dita"/>
  <resource id="7ab81620-9e4a-11e3-9b84-e4ce8f3a81be" relative-uri="black-bird-schematic.jpg"/>
  <resource id="7ab92790-9e4a-11e3-9b84-e4ce8f3a81be" relative-uri="map.ditamap"/>
</dependencies>
```



Retrieve Resource Inbound Links

You use the GET method to retrieve all inbound links for a resource. This is also known as “where used”.

Overview

Type	Value
HTTP Method	GET
URI	<code>/rest/all-files/{resource-id}/links/inbound</code>
Returns	The resource's inbound links

Request Headers

Type	Value	Description
Accept	<code>application/xml</code>	Optional (default)

Response Headers

Type	Value
Content-Type	<code>application/xml; charset=UTF-8</code>



Parameters

No additional parameters are supported.

Example

Request:

```
curl -i -H "Accept: application/xml" -u {your.user@org.com:{token}} https://{your-org}.heretto.com/rest/all-files/8f411bf0-cbba-11e1-ac51-c82a1446d15c/links/inbound
```

Response:

```
HTTP/1.1 200 OK

<?xml version="1.0" ?>
<incoming-links target-resource-id="f1a66c30-a84e-11e6-b8af-12e137855916"
  target-resource-uri="master/open-project/documents/release-notes-warehouse.dita">
  <link source-file-id="9fc43740-a852-11e6-b8af-12e137855916"
    source-file-uri="master/open-project/documents/release-notes.dita"
    link-target="topic-8550/10009"/>
  <link source-file-id="9fc43740-a852-11e6-b8af-12e137855916"
    source-file-uri="master/open-project/documents/release-notes.dita"
    link-target="topic-8550/31811"/>
  <link source-file-id="796d9c30-a852-11e6-b8af-12e137855916"
    source-file-uri="master/open-project/documents/release-notes-conref2.dita"
    link-target="topic-8550/10356"/>
</incoming-links>
```



Folders

Retrieve Folder

You use the GET method to retrieve the folder contents.

Overview

Type	Value
HTTP Method	GET
URI	<code>/rest/all-files/{folder-id}</code>
URI (via repositories)	<code>/rest/branches/{branch-name}/repositories/{repo-id}/documents/{folder-id}</code>
Returns	A representation of the folder and it's contents

Request Headers

Type	Value	Description
Accept	<code>application/xml</code>	Optional (default)



Response Headers

Type	Value
Content-Type	application/xml; charset=UTF-8

Parameters

Type	Value	Description
include-metas	A comma separated list of user-set metadata keys to include on listed resources	?include-metas=status,product

Example

Request:

```
curl -i -H "Accept: application/xml" -u {your.user@org.com:{token}} https://{your-org}.heretto.com/rest/branches/master/repositories/{repo-id}/documents/?include-metas=status
```

Response (listing child collections and resources):

```
HTTP/1.1 200 OK
Content-Type: application/xml; charset=UTF-8
Transfer-Encoding: chunked

<?xml version="1.0" ?>
```



```
<folder id="8f3ed200-cbba-11e1-ac51-c82a1446d15r">
  <name>documents</name>
  <owner>username</owner>
  <group>groupname</group>
  <mode>504</mode>
  <children>
    <folder name="introduction" id="8f3ed200-cbba-11e1-ac51-c82a1446d15w"/>
    <folder name="common" id="23red200-cbba-11e1-ac51-c82a1446d15c"/>
    <resource name="archSpec-base.dita" id="8f3ed200-cbba-11e1-ac51-c82a1446d15c" mime-
type="application/xml" type="Concept" xmldb-type="XMLResource" status="Draft"/>
    <resource name="archSpec-learningTraining.dita" id="8f411bf0-cbba-11e1-ac51-c82a1446d15c" mime-
type="application/xml" type="Concept" xmldb-type="XMLResource" status="In review"/>
    <resource name="archSpec-technicalContent.dita" id="8f425470-cbba-11e1-ac51-c82a1446d15c" mime-
type="application/xml" type="Concept" xmldb-type="XMLResource" status="Approved"/>
  </children>
</folder>
```

Schema

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema" elementFormDefault="qualified">
  <xs:element name="folder">
    <xs:complexType>
      <xs:sequence minOccurs="0">
        <xs:element name="name" type="xs:string"/>
        <xs:element name="owner" type="xs:string"/>
        <xs:element name="group" type="xs:string"/>
        <xs:element name="mode" type="xs:integer"/>
        <xs:element name="children">
          <xs:complexType>
```



```

                                <xs:sequence>
                                    <xs:element maxOccurs="unbounded" minOccurs="0"
ref="folder"/>
                                    <xs:element maxOccurs="unbounded" minOccurs="0"
ref="resource"/>
                                </xs:sequence>
                            </xs:complexType>
                        </xs:element>
                    </xs:sequence>
                    <xs:attribute name="id" use="required"/>
                </xs:complexType>
            </xs:element>

<xs:element name="resource">
    <xs:complexType>
        <xs:attribute name="id" use="required"/>
        <xs:attribute name="name" use="required" type="xs:string"/>
        <xs:attribute name="xml:db-type" use="required">
            <xs:simpleType>
                <xs:restriction base="xs:string">
                    <xs:enumeration value="XMLResource"/>
                    <xs:enumeration value="BinaryResource"/>
                </xs:restriction>
            </xs:simpleType>
        </xs:attribute>
        <xs:attribute name="locked-by" use="optional" type="xs:string"/>
        <xs:attribute name="type" use="required" type="xs:string"/>
        <xs:attribute name="mime-type" use="required" type="xs:string"/>
        <xs:anyAttribute/>
    </xs:complexType>

```



```
</xs:element>  
</xs:schema>
```



Create Folder Method

You use the POST method to create folders in the repository.

Overview

Type	Value
HTTP Method	POST
URI	<code>/rest/all-files/{parent-folder-id}</code>
URI (Via repositories)	<code>/rest/branches/{branch-name}/repositories/{repo-id}/documents/{parent-folder-id}</code>
Returns	Empty or <code>201 CREATED</code> if created

Request Headers

No request headers are required.

POST body

The POST body takes the following format:



```
<folder>
  <name>{new-collection-path}</name>
</folder>
```

The `<name>` field is required. It may be either a simple name or a path with multiple segments, in this case each segment will be created, but information will only be return on the final segment.

Response Headers

No additional headers are returned.

Parameters

No additional parameters are supported.

Example

Request (creating a new folder in a current folder):

```
curl -i -H "Content-Type: application/xml" -u {your.user@org.com:{token}} -X POST -d "<folder><name>new-
folder</name></folder>" https://{your-org}.heretto.com/rest/all-files/75927d70-9e4a-11e3-9b84-e4ce8f3a81be
```

Response:

```
HTTP/1.1 201 Created

<?xml version="1.0" ?>
<folder id="122b77e0-a085-11e3-9478-e4ce8f3a81be">
  <name>new-folder</name>
  <xmldb-uri>product-documentation/documents/Demo/new-folder</xmldb-uri>
  <owner>user.name@domain.com</owner>
```



```
<group>__demo</group>  
<mode>504</mode>  
<children/>  
</folder>
```



Locks

Retrieve Lock

You use the GET method to retrieve the current lock (if any) on a resource.

Overview

Type	Value
HTTP Method	GET
URI	/rest/all-files/{resource-id}/lock
Returns	The current lock on the resource (if any)

Request Headers

Type	Value	Description
Accept	application/xml	Optional (default)

Response Headers

Type	Value
Content-Type	application/xml; charset=UTF-8



Parameters

No additional parameters are supported.

Example

Request:

```
curl -i -H "Accept: application/xml" -u {your.user@org.com:{token}} https://{your-org}.heretto.com/rest/
repositories/{repo-id}/documents/8f411bf0-cbba-11e1-ac51-c82a1446d15c/lock
```

Response (when a user has a lock):

```
HTTP/1.1 200 OK
Content-Type: application/xml; charset=UTF-8
Transfer-Encoding: chunked

<?xml version="1.0" ?>
<lock locking-user="admin" token="jorsek-checked-out-lock:2013-02-04T11:59:27.742-05:00#c3799eee-c9ab-4a5a-
a654-801dba0d81cd" />
```

Response (when no user has a lock):

```
HTTP/1.1 200 OK
Content-Type: application/xml; charset=UTF-8
Transfer-Encoding: chunked

<?xml version="1.0" ?>
<lock/>
```



Add lock

You use the POST method to add a lock on a resource.

Overview

Type	Value
HTTP Method	POST
URI	/rest/all-files/{resource-id}/lock
Returns	200 OK if successful

Request Headers

Type	Value	Description
Content-Type	application/xml	Required

Response Headers

Type	Value
Content-Type	text; charset=UTF-8



Parameters

No additional parameters are supported.

POST body

The POST body takes the following format:

```
<lock locking-user="john@doe.com" />
```

Where the `locking-user` is the username/email address of the user who will own this lock.

Example

Request:

```
curl -i -H "Content-Type: application/xml" -X POST -d "<lock locking-user='{john@doe.com}' />" -u  
{your.user@org.com:{token}} https://{your-org}.heretto.com/rest/all-files/8f3ed200-cbba-11e1-ac51-c82a1446d15c/  
lock
```

Response (`200 OK` when successful):

```
HTTP/1.1 200 OK
```



Remove Lock

You use the DELETE method to remove any current locks on a resource.

Overview

☆ **IMPORTANT:** This method is applied to the entire “lock” collection. Since there can never be more than a single lock, there is no need to supply a specific lock ID to delete.

Type	Value
HTTP Method	DELETE
URI	/rest/all-files/{resource-id}/lock
Returns	200 OK if successful

Example

Request (removing the existing lock):

```
curl -i -X DELETE -u {your.user@org.com:{token}} https://{your-org}.heretto.com/rest/repositories/repo1/documents/8f411bf0-cbba-11e1-ac51-c82a1446d15c/lock
```

Response:

```
HTTP/1.1 200 OK
```



Branches and Releases

Retrieve Branches

You use the GET method to retrieve the current branches in the system.

Overview

Type	Value
HTTP Method	GET
URI	/rest/branches/
Returns	A representation of the branch and the repositories it contains

Request Headers

Type	Value	Description
Accept	application/xml	Optional (Default)

Response Headers

Type	Value
Content-Type	application/xml; charset=UTF-8



Parameters

No additional parameters are supported.

Example

Request:

```
curl -i -H "Accept: application/xml" -u {your.user@org.com:{token}} https://{your-org}.heretto.com/rest/branches/
```

Response (listing all branches and repositories):

```
HTTP/1.1 200 OK
Content-Type: application/xml; charset=UTF-8
Transfer-Encoding: chunked

<?xml version="1.0" ?>
<branches>
  <branch name="test">
    <repository name="demo"/>
  </branch>
  <branch name="master">
    <repository name="demo"/>
    <repository name="documentation"/>
  </branch>
</branches>
```

ⓘ NOTE: By default there will always be a “master” branch, even if you have not enabled Branching or created any other branches.



Retrieve Releases

You use the GET method to retrieve non-deprecated releases of a resource from your Content Library.

Overview

HTP Method	GET
URI	<code>/rest/all-files/{uuid}/releases</code> Where, <code>{uuid}</code> is the UUID of the resource.
Returns	A representation of non-deprecated releases of the resource. ① NOTE: Releases from every branch are included. For example, if a released map exists on two branches, two identical responses will be returned for each map on each branch.

Request Headers

Type	Value	Description
Accept	<code>application/xml</code>	Optional (Default)



Response Headers

Type	Value
Content-Type	application/xml; charset=UTF-8

Parameters

withDeprecated	Defaults to <code>false</code> . When <code>true</code> , includes deprecated releases.	<code>?withDeprecated="true"</code>
----------------	---	-------------------------------------

Example

Request:

```
curl -i -H "Accept: application/xml" -u {your.user@org.com:{token}} https://{your-org}.heretto.com/rest/{uuid}/releases
```

Response:

```
{
  "releaseByFiles": {
    "e1f91d23-3d14-43ef-a4e1-7da5c4bcce94": {
      "@class": "com.jorsek.api.shared.files.impl.XMLDocImpl",
      "name": "m1.ditamap",
      "ownerUser": "user.name@domain.com",
      "ownerGroup": "heretto",
      "mode": "rwxrwx---",
      "parentId": "d0f69340-3e48-11eb-b676-0242f2c6418c",
      "mimeType": "application/xml",
    }
  }
}
```



```
"hasExtendedPermissions": false,
"canRead": true,
"canWrite": true,
"canExecute": true,
"metadata": {
  "@class": "com.jorsek.api.shared.metadata.impl.MetadataImpl",
  "data": {
    "validation-err-msg": "",
    "lock-owner": "",
    "normalized-checksum": "19a7491e220f63249f06066fce1db4c2",
    "Product_Tier": "",
    "title": "m1",
    "Test_Label": "",
    "num-open-comments": "0",
    "Feeling": "",
    "__has_broken_links": "false",
    "Version": "",
    "dita-domains": "",
    "checksum": "19a7491e220f63249f06066fce1db4c2",
    "original-template-label": "Default Map",
    "Defect": "",
    "content-type": "Map",
    "__checksum-dirty": "false",
    "document-valid-md-field": "true",
    "__release-id": "e1f91d23-3d14-43ef-a4e1-7da5c4bcce94",
    "heretto": "",
    "__in_a_release": "true",
    "Pass_or_Fail": "",
    "Automatic_Upload": "",
    "char-count": "0",
    "is-valid": "true",
```



```
"Color": "",
"__source-resource-uuid": "c4c9d0a0-3e48-11eb-b676-0242f2c6418c",
"dita-class": "- map/map ",
"original-template-name": "map-template.ditamap",
"__document_owner": "name.surname@heretto.com",
"Subject": "",
"tags": "",
"marketing": "",
"Test_Label_2": "",
"__document_links_last_processed_rev": "-1",
"__last_modified_revision": "-1",
"Rebranding_Metadata": "",
"word-count": "0",
"Stephani_Test_1": "",
"Test_Date_1": "",
"Stephani_Test_2": "",
"__root-resource-uuid": "c4c9d0a0-3e48-11eb-b676-0242f2c6418c",
"Interface": "",
"__last_modified_by": "SYSTEM",
"status": "in_progress",
"Test_Date_2": ""
}
},
"lastModified": 1607986175716,
"created": 1607976802996,
"lockingUsername": null,
"contentTypeName": "Map",
"lockToken": null,
"ID": "d0f6e160-3e48-11eb-b676-0242f2c6418c",
"URI": "/db/organizations/heretto/releases/e1f91d23-3d14-43ef-a4e1-7da5c4bcce94/master/test/
documents/_Personal_Spaces/Name_Surname/2/m1.ditamap"
```



```
    }
  },
  "releases": [
    {
      "@class": "com.jorsek.api.shared.release.ReleaseImpl",
      "id": "e1f91d23-3d14-43ef-a4e1-7da5c4bcce94",
      "rootResourceId": "c4c9d0a0-3e48-11eb-b676-0242f2c6418c",
      "sourceResourceId": "c4c9d0a0-3e48-11eb-b676-0242f2c6418c",
      "branchOfOriginId": "8bdefdd0-8f66-11e3-a2d7-bc764e113e3b",
      "completedDateTime": "2020-12-14T20:13:23.242Z",
      "depreciatedDateTime": null,
      "createdBy": "name.name@domain.com",
      "rev": "-1",
      "name": "abc",
      "branchOfOriginName": "master",
      "notes": "",
      "pathToReleasedResources": "/db/organizations/heretto/releases/e1f91d23-3d14-43ef-a4e1-7da5c4bcce94/
master/test/documents/_Personal_Spaces/Name_Surname/2/m1.ditamap"
    }
  ]
}
```



Search

You use the POST method to search for files and/or folders in the repository.

Overview

Type	Value
HTTP Method	POST
URI	/ezdnxtgen/api/search
Returns	Search results as JSON

Request Headers

Type	Value	Description
Content-Type	A valid content type to store the document as.	Required - Must be set to application/xml
Content-Length	The length/size of the content to be stored	Optional but recommended - If not supplied the system will attempt to determine the content length

POST body

The POST body takes a JSON search request object. This object has the following parameters:



queryString

Optional - A string parsed by the Lucene query parser. This should follow the [Lucene Query Parser Syntax](#).

drilldowns

Optional - An array containing arrays which contain arrays of strings. The innermost arrays are paths specifying a metadata selection. The outermost arrays are AND-ed with each other, while the inner arrays are OR-ed with each other.

For example, to search for topics with an Approved status, the drill-downs would be:

```
[ [ [ "status", "approved" ] ] ]
```

To search for files with a status of Approved or Draft, the drill-downs would be:

```
[ [ [ "status", "approved" ], [ "status", "draft" ] ] ]
```

To search for files with a status of Approved or Draft and a Content Type of Map, the drill-downs would be:

```
[ [ [ "status", "approved" ], [ "status", "draft" ] ], [ [ "content-type", "map" ] ] ]
```

queryFilters

Optional - An array of strings to filter the result set.

queryFilters can be used to search for date ranges. To do this, specify the query in this format: "{Field}{Operator}{Epoch}" where Field is the field you are searching, Operator is one of <, <=, >, >=, and Epoch is Epoch milliseconds. For example, to search for files modified between January 1 2019 and January 7 2019, add this to your queryFilter:

```
["last_modified>1546304400000", "last_modified<1546822800000"]
```

① NOTE: Out of the box date fields are `last_modified` and `created`.



foldersToSearch

Required - A map from string to Boolean that specifies the folders to search. Keys are folder URIs. Values are a Boolean. "True" specifies including child folders. "False" specifies excluding child folders.

facetRequests

Optional - An array of array of strings. Inner arrays are paths specifying facet results for the response. For example, if you want facet results for Status, `facetRequest` would be `[ [ "status" ] ]`.

startOffset

Optional - The start offset for the result set. Defaults to 0.

endOffset

Optional - The end offset for the result set. Defaults to 100.

searchResultType

Required - A string specifying the type of results. Use `FILES_ONLY` to only return files (most common), `FOLDERS_ONLY` to only return folders, or `FILES_AND_FOLDERS` to return both files and folders.

ⓘ NOTE: Folders will always be returned at the top of the result list.



searchResultOptions

A map specifying options for the results.

markIfInDependencySets

Optional - An array of strings. Each string is a map UUID. If a result is contained in any of these maps, the `isDependencyOfProvidedSet` in the searchHit in the results will be set to `true`. This tracks explicit dependencies only.

markIfInFullDependencySets

Optional - An array of strings. Each string is a map UUID. If a result is contained in any of these maps, the `isDependencyOfProvidedFullSet` in the searchHit in the results will be set to `true`. This tracks full dependencies, meaning explicit and implicit dependencies. Links between html files are also tracked by full dependencies.

includeContextMatches

Optional - A boolean specifying if results should contain the `matchHtml` property.

includeIncomingLinks

Optional - A boolean specifying if results should include the `numIncomingLinks` property.



sort

Optional. A string specifying how results are sorted. Possible values are:

- RELEVANCE (default)
- FILENAME_ASC
- FILENAME_DESC
- LAST_MODIFIED_ASC
- LAST_MODIFIED_DESC
- TITLE_ASC
- TITLE_DESC
- CREATED_ASC
- CREATED_DESC
- CONTENT_TYPE_ASC
- CONTENT_TYPE_DESC
- STATUS_ASC
- STATUS_DESC

containedInMapFilterIds

Optional - An array of strings. Each string is a map UUID.

mapToDitavalsFilter

Optional - A map of string to array of strings. The key is a ditamap UUID. The array is an optional list of ditavals. Ditavals can be specified as xml, or can be a UUID of a ditaval file.



addPrefixAndFuzzy

Optional - A boolean specifying to expand the `queryString` with prefix and fuzzy matching. Default to true.

ⓘ NOTE: The query will only expand if `queryString` contains no special Lucene characters.

includeBreadcrumbs

Optional - A boolean specifying whether or not to include breadcrumbs to the maps in `containedInMapFilterIds`. Default to false.

Response Headers

No additional headers are returned.

Response Body

The response body is a json object. This object has the following parameters:

originalRequest

The search request.

totalResults

A number specifying total number of results this query matched.

**hits**

An array of objects representing the search hits, defined as follows.

fileEntity

A JSON representation of the file containing URI, UUID, and file metadata. See examples below.

isDependencyOfProvidedSet

A Boolean. True if this file is contained in the search requests's **markIfInDependencySets**.

numIncomingLinks

A number specifying the number of incoming links

Parameters

No additional parameters are supported.

Example: Basic search

Request:

```
curl 'https://localhost:8080/ezdnxtgen/api/search' -u admin -H 'Accept: application/json' -H 'Content-Type: application/json; charset=utf-8' -d '{"foldersToSearch":{"db/organizations/test-org/repositories/project-three/documents/search-test/":true}, "startOffset":0, "endOffset":20, "searchResultType":"FILES_ONLY"}'
```

Response (two hits returned for the files "a.dita" and "b.dita"):

```
{
  "originalRequest": {
    "queryString": "",
    "drilldowns": [],
    "queryFilters": [],
    "foldersToSearch": {
```



```
    "/db/organizations/test-org/repositories/project-three/documents/search-test/": true
  },
  "facetRequests": [],
  "startOffset": 0,
  "endOffset": 20,
  "searchResultType": "FILES_ONLY",
  "searchResultOptions": {
    "markIfInDependencySets": [],
    "includeContextMatches": true,
    "includeIncomingLinks": false
  },
  "sort": "RELEVANCE",
  "grouping": null,
  "containedInMapFilterIds": [],
  "addPrefixAndFuzzy": true,
  "includeBreadcrumbs": false
},
"totalResults": 2,
"hits": [
  {
    "fileEntity": {
      "@class": "com.jorsek.api.shared.files.impl.XMLDocImpl",
      "name": "a.dita",
      "ownerUser": "admin",
      "ownerGroup": "__test-org-project-three",
      "mode": "rwxrwx---",
      "parentId": "9bf2b6b0-5f31-11e6-885b-62da9cd96c9d",
      "mimeType": "application/xml",
      "hasExtendedPermissions": false,
      "metadata": {
        "@class": "com.jorsek.api.shared.metadata.impl.MetadataImpl",
        "data": {
          "validation-err-msg": "",
          "char-count": "42",
          "lock-owner": "",
          "normalized-checksum": "21a987107ceef04703f41bc4c70116d4",
          "is-valid": "true",
```



```

    "dita-class": "- topic/topic ",
    "title": "a",
    "__document_owner": "admin",
    "tags": "",
    "num-open-comments": "0",
    "__document_links_last_processed_rev": "-1",
    "__last_modified_revision": "-1",
    "__has_broken_links": "false",
    "dita-domains": "",
    "checksum": "dc046748b77852bdd7d4af3fe04f7e70",
    "content-type": "Topic",
    "word-count": "3",
    "document-valid-md-field": "true",
    "__last_modified_by": "admin",
    "status": ""
  }
},
"lastModified": 1470857744499,
"created": 1470857743670,
"lockingUsername": null,
"contentTypeName": "Topic",
"lockToken": "",
"ID": "a0c5c290-5f31-11e6-885b-62da9cd96c9d",
"URI": "/db/organizations/test-org/repositories/project-three/documents/search-test/a.dita"
},
"isDependencyOfProvidedSet": false,
"numIncomingLinks": 0,
"matchHtml": ""
},
{
  "fileEntity": {
    "@class": "com.jorsek.api.shared.files.impl.XMLDocImpl",
    "name": "b.dita",
    "ownerUser": "admin",
    "ownerGroup": "__test-org-project-three",
    "mode": "rwxrwx---",
    "parentId": "9bf2b6b0-5f31-11e6-885b-62da9cd96c9d",

```



```

"mimeType": "application/xml",
"hasExtendedPermissions": false,
"metadata": {
  "@class": "com.jorsek.api.shared.metadata.impl.MetadataImpl",
  "data": {
    "validation-err-msg": "",
    "char-count": "42",
    "normalized-checksum": "21a987107ceef04703f41bc4c70116d4",
    "is-valid": "true",
    "dita-class": "- topic/topic ",
    "title": "b",
    "__document_owner": "admin",
    "tags": "",
    "num-open-comments": "0",
    "__document_links_last_processed_rev": "-1",
    "__last_modified_revision": "-1",
    "__has_broken_links": "false",
    "dita-domains": "",
    "checksum": "dc046748b77852bdd7d4af3fe04f7e70",
    "content-type": "Topic",
    "word-count": "3",
    "document-valid-md-field": "true",
    "__last_modified_by": "admin",
    "status": "approved"
  }
},
"lastModified": 1470861717715,
"created": 1470861717078,
"lockingUsername": null,
"contentTypeName": "Topic",
"lockToken": "",
"ID": "e11c9ef0-5f3a-11e6-bf5b-62da9cd96c9d",
"URI": "/db/organizations/test-org/repositories/project-three/documents/search-test/b.dita"
},
"isDependencyOfProvidedSet": false,
"numIncomingLinks": 0,
"matchHtml": ""

```



```
}
],
"facetResults": {},
"grouping": null
}
```

Example: Search for Status Approved

Request:

```
curl 'https://localhost:8080/ezdnextgen/api/search' -u admin -H 'Accept: application/json' -H 'Content-Type: application/json; charset=utf-8' -d '{"foldersToSearch":{"db/organizations/test-org/repositories/project-three/documents/search-test/":true}, "drilldowns": [ [ ["status", "approved"] ] ], "startOffset":0, "endOffset":20, "searchResultType":"FILES_ONLY"}'
```

Response (one hit returned for the **b.dita** file):

```
{
  "originalRequest": {
    "queryString": "",
    "drilldowns": [
      [
        [
          "status",
          "approved"
        ]
      ]
    ],
    "queryFilters": [],
    "foldersToSearch": {
      "db/organizations/test-org/repositories/project-three/documents/search-test/": true
    },
    "facetRequests": [],
    "startOffset": 0,
    "endOffset": 20,

```



```
"searchResultType": "FILES_ONLY",
"searchResultOptions": {
  "markIfInDependencySets": [],
  "includeContextMatches": true,
  "includeIncomingLinks": false
},
"sort": "RELEVANCE",
"grouping": null,
"containedInMapFilterIds": [],
"addPrefixAndFuzzy": true,
"includeBreadcrumbs": false
},
"totalResults": 1,
"hits": [
  {
    "fileEntity": {
      "@class": "com.jorsek.api.shared.files.impl.XMLDocImpl",
      "name": "b.dita",
      "ownerUser": "admin",
      "ownerGroup": "__test-org-project-three",
      "mode": "rwxrwx---",
      "parentId": "9bf2b6b0-5f31-11e6-885b-62da9cd96c9d",
      "mimeType": "application/xml",
      "hasExtendedPermissions": false,
      "metadata": {
        "@class": "com.jorsek.api.shared.metadata.impl.MetadataImpl",
        "data": {
          "validation-err-msg": "",
          "char-count": "42",
          "normalized-checksum": "21a987107ceef04703f41bc4c70116d4",
          "is-valid": "true",
          "dita-class": "- topic/topic ",
          "title": "b",
          "__document_owner": "admin",
          "tags": "",
          "num-open-comments": "0",
          "__document_links_last_processed_rev": "-1",
```



```

    "__last_modified_revision": "-1",
    "__has_broken_links": "false",
    "dita-domains": "",
    "checksum": "dc046748b77852bdd7d4af3fe04f7e70",
    "content-type": "Topic",
    "word-count": "3",
    "document-valid-md-field": "true",
    "__last_modified_by": "admin",
    "status": "approved"
  }
},
"lastModified": 1470861717715,
"created": 1470861717078,
"lockingUsername": null,
"contentTypeName": "Topic",
"lockToken": "",
"ID": "e11c9ef0-5f3a-11e6-bf5b-62da9cd96c9d",
"URI": "/db/organizations/test-org/repositories/project-three/documents/search-test/b.dita"
},
"isDependencyOfProvidedSet": false,
"numIncomingLinks": 0,
"matchHtml": ""
}
],
"facetResults": {},
"grouping": null
}

```

Example: Search with facet requests

Request:

```

curl 'https://localhost:8080/ezdnxtgen/api/search' -u admin -H 'Accept: application/json' -H 'Content-Type: application/json; charset=utf-8' -d '{"foldersToSearch":{"db/organizations/test-org/repositories/project-three/documents/search-test/":true}, "facetRequests": [ ["status"] ], "startOffset":0, "endOffset":20, "searchResultType":"FILES_ONLY"}'

```



Response (two hits with facets results showing one file has Status Approved):

```
{
  "originalRequest": {
    "queryString": "",
    "drilldowns": [],
    "queryFilters": [],
    "foldersToSearch": {
      "/db/organizations/test-org/repositories/project-three/documents/search-test/": true
    },
    "facetRequests": [
      [
        "status"
      ]
    ],
    "startOffset": 0,
    "endOffset": 20,
    "searchResultType": "FILES_ONLY",
    "searchResultOptions": {
      "markIfInDependencySets": [],
      "includeContextMatches": true,
      "includeIncomingLinks": false
    },
    "sort": "RELEVANCE",
    "grouping": null,
    "containedInMapFilterIds": [],
    "addPrefixAndFuzzy": true,
    "includeBreadcrumbs": false
  },
  "totalResults": 2,
  "hits": [
    {
      "fileEntity": {
        "@class": "com.jorsek.api.shared.files.impl.XMLDocImpl",
        "name": "a.dita",
        "ownerUser": "admin",
```



```

"ownerGroup": "__test-org-project-three",
"mode": "rwxrwx---",
"parentId": "9bf2b6b0-5f31-11e6-885b-62da9cd96c9d",
"mimeType": "application/xml",
"hasExtendedPermissions": false,
"metadata": {
  "@class": "com.jorsek.api.shared.metadata.impl.MetadataImpl",
  "data": {
    "validation-err-msg": "",
    "char-count": "42",
    "lock-owner": "",
    "normalized-checksum": "21a987107ceef04703f41bc4c70116d4",
    "is-valid": "true",
    "dita-class": "- topic/topic ",
    "title": "a",
    "__document_owner": "admin",
    "tags": "",
    "num-open-comments": "0",
    "__document_links_last_processed_rev": "-1",
    "__last_modified_revision": "-1",
    "__has_broken_links": "false",
    "dita-domains": "",
    "checksum": "dc046748b77852bdd7d4af3fe04f7e70",
    "content-type": "Topic",
    "word-count": "3",
    "document-valid-md-field": "true",
    "__last_modified_by": "admin",
    "status": ""
  }
},
"lastModified": 1470857744499,
"created": 1470857743670,
"lockingUsername": null,
"contentTypeName": "Topic",
"lockToken": "",
"ID": "a0c5c290-5f31-11e6-885b-62da9cd96c9d",
"URI": "/db/organizations/test-org/repositories/project-three/documents/search-test/a.dita"

```



```

    },
    "isDependencyOfProvidedSet": false,
    "numIncomingLinks": 0,
    "matchHtml": ""
  },
  {
    "fileEntity": {
      "@class": "com.jorsek.api.shared.files.impl.XMLDocImpl",
      "name": "b.dita",
      "ownerUser": "admin",
      "ownerGroup": "__test-org-project-three",
      "mode": "rwxrwx---",
      "parentId": "9bf2b6b0-5f31-11e6-885b-62da9cd96c9d",
      "mimeType": "application/xml",
      "hasExtendedPermissions": false,
      "metadata": {
        "@class": "com.jorsek.api.shared.metadata.impl.MetadataImpl",
        "data": {
          "validation-err-msg": "",
          "char-count": "42",
          "normalized-checksum": "21a987107ceef04703f41bc4c70116d4",
          "is-valid": "true",
          "dita-class": "- topic/topic ",
          "title": "b",
          "__document_owner": "admin",
          "tags": "",
          "num-open-comments": "0",
          "__document_links_last_processed_rev": "-1",
          "__last_modified_revision": "-1",
          "__has_broken_links": "false",
          "dita-domains": "",
          "checksum": "dc046748b77852bdd7d4af3fe04f7e70",
          "content-type": "Topic",
          "word-count": "3",
          "document-valid-md-field": "true",
          "__last_modified_by": "admin",
          "status": "approved"
        }
      }
    }
  }
}

```



```
    }
  },
  "lastModified": 1470861717715,
  "created": 1470861717078,
  "lockingUsername": null,
  "contentTypeName": "Topic",
  "lockToken": "",
  "ID": "e11c9ef0-5f3a-11e6-bf5b-62da9cd96c9d",
  "URI": "/db/organizations/test-org/repositories/project-three/documents/search-test/b.dita"
},
"isDependencyOfProvidedSet": false,
"numIncomingLinks": 0,
"matchHtml": ""
}
],
"facetResults": {
  "status/approved": 1
},
"grouping": null
}
```



Webhooks

You can send webhook notifications from Heretto CCMS that can trigger various actions on external servers. External servers can be configured to pull content from the CCMS on a webhook notification.

Webhooks can be configured for all cloud publishing engines that are an integral part of the CCMS..

① **NOTE:** This functionality may not be available by default in Heretto CCMS. For more information, contact your Customer Success Manager.

By default, when the CCMS sends a webhook notification, it waits for up to three minutes to get a response. If it receives no response, it reports an error with the target server. This webhook timeout is configurable. For details, contact your Customer Success Manager.

Webhook notifications on publish

You can send webhook notifications automatically on successful publishing jobs or on any publishing job.

Options

Webhooks Do not send

Debug publish

Do not send

Send on success

Send regardless of build result

Do not send

Webhook notification is not sent.



Send on success

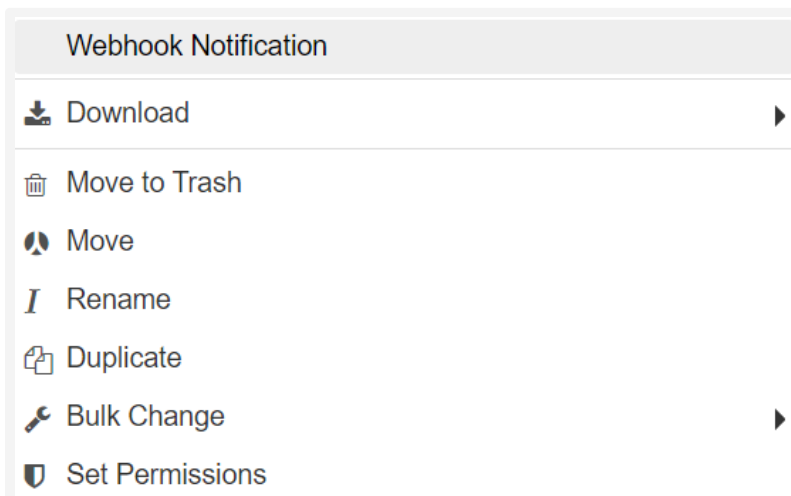
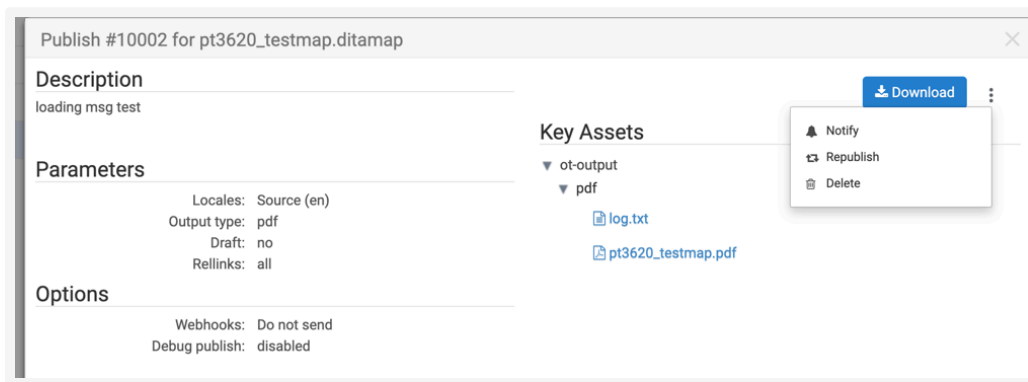
Webhook notification is sent after a publishing job is completed successfully.

Send regardless of build result

Webhook notification is sent for both successful and unsuccessful publishing jobs.

Webhook notifications on demand

You can manually send webhook notifications for past publishing jobs and resources in your Content Library.





Triggering Actions on External Servers with Webhooks (Notify)

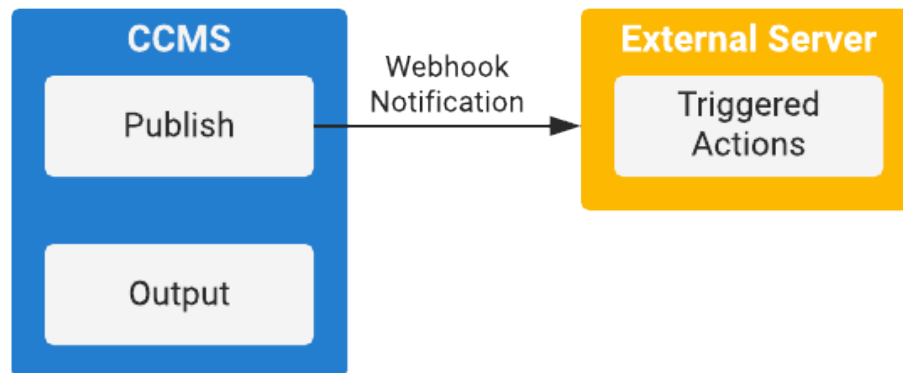
You can use webhooks notifications to trigger various actions on servers external to Heretto CCMS.

Heretto CCMS can be configured to send webhook notifications about publishing events to external servers. On a webhook notification, the external server can trigger various actions. For example, the external server can automatically send emails on successful publishing jobs.

Example

In this example, a publishing job is initiated in Heretto CCMS. The publishing job generates output in Heretto CCMS. If the publishing job is successful, then the CCMS sends a webhook notification to the external server. The external server triggers actions based on the webhook notification.

Figure 1. Webhook Notification Process

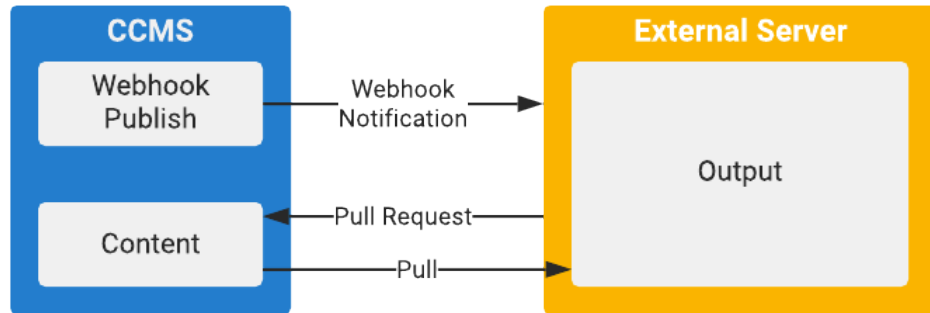




Pulling Content from Heretto CCMS to External Servers (NotifyPull)

External servers may be configured to pull content from Heretto CCMS on a webhook notification.

The majority of output formats produced by the DITA Open Toolkit (DITA-OT) publishing engine¹ can be pulled from Heretto CCMS to an external server.



1. For more information about the DITA-OT plugins, see <https://www.dita-ot.org/plugins>.

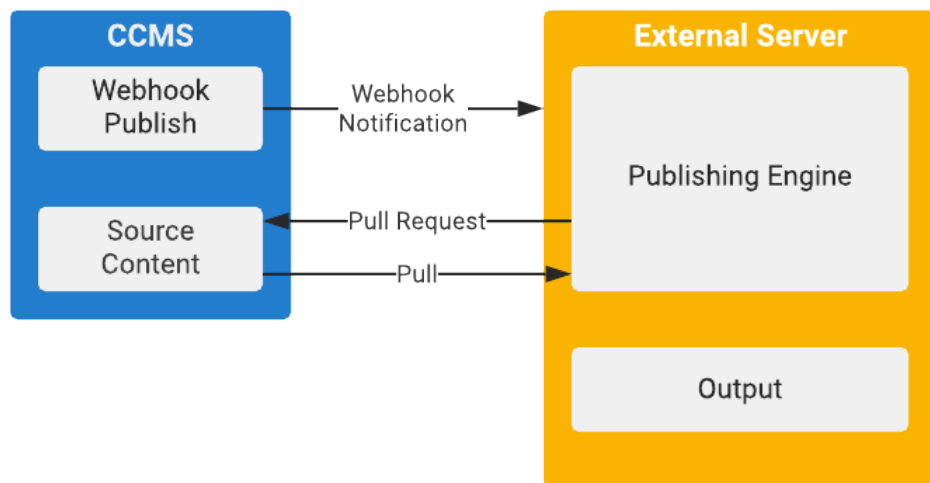


Publishing Content on External Servers with Webhooks (NotifyPull)

External servers may be configured to pull DITA content from Heretto CCMS on a webhook notification and transform the pulled content. Sending a webhook notification for source content is useful if you want to process DITA content by using a publishing engine hosted on an external server.

In the following example, a webhook notification is sent from Heretto CCMS. The external server triggers a publishing engine that pulls and transforms the source content from Heretto CCMS. The output is generated on the external server side.

Figure 2. External Publishing Process with Webhooks





Implement a Webhook Integration

To implement a webhook integration, Heretto CCMS requires only a few endpoints to be available.

Before you begin

Ensure that:

- You are a Heretto CCMS administrator
- You have access to the target system
- You read through [Webhook Authentication and Security](#) on page 74

Procedure

- 1 By using Oxygen, configure webhooks on your Heretto CCMS instance. See [Webhook Configuration Files](#) on page 76.

① NOTE: To complete this step, you may need to contact your Customer Success Manager.

- 2 Implement the `listen_url`. See [Webhook Notification Request](#) on page 80.
- 3 If needed, configure the pull request pipeline. See [Get Document Bundle](#) on page 24 or [Get Published Document](#) on page 26.

Next steps

Test your integration by doing any of the following:

- [Send a Webhook Notification on Publish](#) on page 88
- [Send a Webhook Notification on Demand](#) on page 91



Webhook Authentication and Security

Webhooks do not require authentication, but there are additional security measures available for webhook requests to suit your organization's requirements. Webhooks use the internal XQuery Authentication Utils.

No Authentication

No authentication is an option for webhook requests. In this case, webhooks are sent to the provided URLs in the target system. No authentication headers are included in the webhook request to provide extra layers of security when an event is triggered.

HTTP Header Authentication

You can configure webhooks to include HTTP headers in the request. You can use this to configure Basic authentication or Bearer authentication.

For Basic authentication, you configure a username and password for the target system, and the Basic authentication fields are included in the header of the HTTP webhook request.

Bearer authentication is generally used for OAuth but can also be used for stored key authentication.

OAuth 2.0

You can configure webhooks to fetch an access token using the client credentials OAuth method before sending the webhook request. This access token is then included as an HTTP header of the webhook request.

To configure OAuth using client credentials authentication, you must provide the following:

- Client ID
- Client Secret
- Token URL (this is a URL of an authorization service at the target service)



Request Signature Header

For additional security, to each webhook request header, Heretto CCMS adds an SHA-256 hash computed on the basis of the request properties. To harden the webhooks security, you can define a secret that will be taken into account during the hash computation.

① **NOTE:** For more information, contact your Customer Success Manager.



Webhook Configuration Files

Webhook configurations define the target system to be notified.

ⓘ NOTE: You may need to contact your Customer Success Manager to access and modify the configuration files.

Configuration file

Configuration file location: `/db/organizations/{org-name}/configuration/webhooks/config.xml`

Figure 3. Configuration file example

```
<webhook_config id="webhook_name">
  <name>My Webhook</name>
  <description>This is an example webhook</description>
  <auth auth-config="../../auth-configs/client-credentials-auth.xml"/>
  <notify_url>https://target.system.com/listening/uri</notify_url>
  <!-- optional -->
  <status_url>https://target.system.com/status/uri</status_url>
  <publish_form_url>https://target.system.com/publish/form/uri</publish_form_url>
  <secret>my_secret</secret> <!-- optional -->
  <events>
    <event id="dita-ot-publish-complete"/>
    <event id="pdf-generator-publish-complete"/>
  </events>
  <filters> </filters>
</webhook_config>
```

You can specify any valid value for the target notify URL, configuration file name, and the `webhook_config @id` attribute. The administrators of the target system should provide the notify URL.

For webhook testing, you can use this URL:



```
<notify_url>https://easydita.requestcatcher.com/</notify_url>
```

① NOTE: When testing the notification configuration, be sure to specify an event, as shown in the [Figure 3. Configuration file example on page 76](#).

Publishing interface configuration

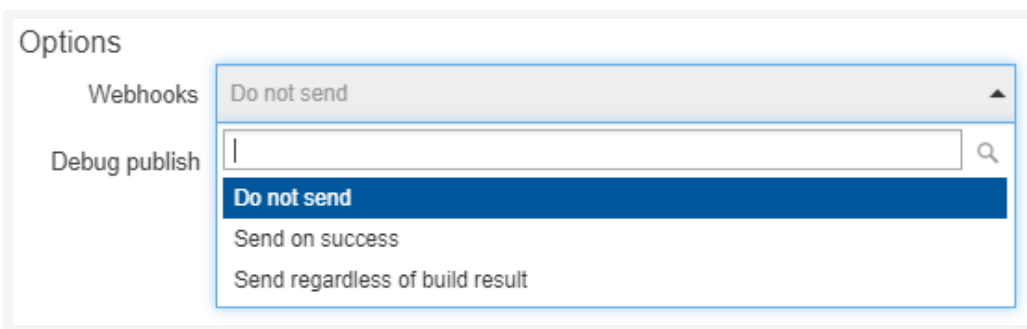
To use the Webhook functionality, the following feature flags must be enabled for your organization:

```
<settings>  
  <set id="feature.publish.notify-select" value="true"/>  
  <set id="feature.publish.manual-notify-button" value="true"/>  
</settings>
```

The feature flags are set in the `/db/organizations/{your-org}/.organization-settings.xml` file.

feature.publish.notify-select

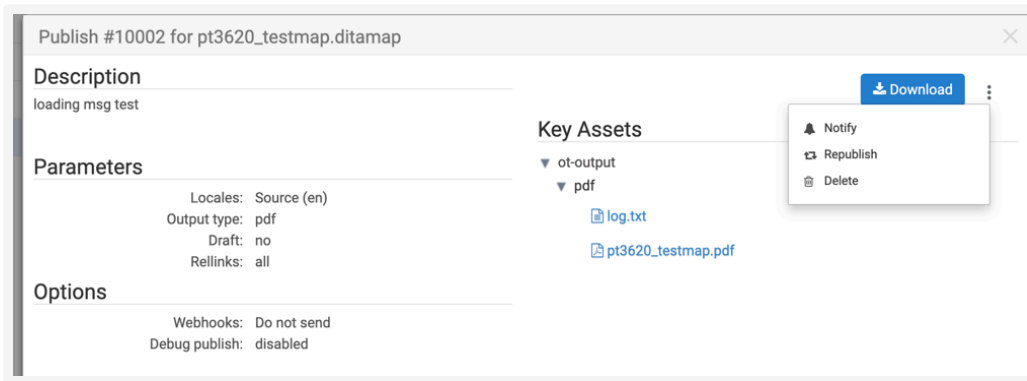
Enables the webhooks drop-down menu in the Publishing interface. Available only for DITA-OT scenarios.





feature.publish.manual-notifiy-button

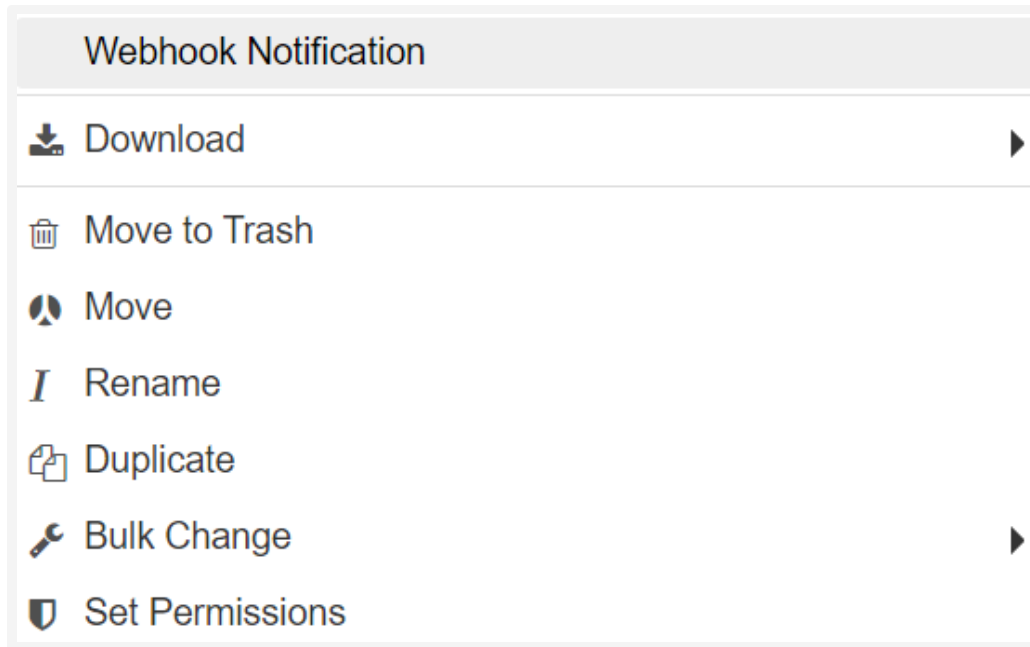
Enables the **Notify** option within publish records. The **Notify** option can be found within the **More Options** menu in a publish record.



Context (right-click) menu configuration

To add an option to the right-click menu, add the following into the `/db/organizations/{your-org}/.organization-settings.xml` file:

```
<custom-context-menu-reports>
  <custom-context-menu-report uri="/db/organizations/{org-name}/configuration/xquery/
simple_webhook_notification.xql" label="Webhook Notification"/>
</custom-context-menu-reports>
```



To modify the XQL script that triggers the webhook notification, edit the following file: `/db/organizations/{org-name}/configuration/xquery/simple_webhook_notification.xql`



Webhook Notification Request

The notification request is a request sent from Heretto CCMS to the target system's specified `listen_url`.

A `listen_url` is a web endpoint that triggers a workflow when it receives a request. This is exactly the same as a `listen_url` for a webhook. In fact, the NotifyPull system uses Heretto CCMS webhook request system to send the notification request to the `listen_url`.

Overview

Type	Value
HTTP Method	POST
URI	Specified in the configuration for the webhook
Returns	Empty

Request Headers

Type	Value	Description
Authorization (see Webhook Configuration on page 76)	<code>{configured-HTTP-header}</code>	<code>{stored-key-value}</code>



Response Headers

Type	Value
Status	200 OK if successfully received

Parameters

event_id

The event name for this notification event.

event_data

Extra data that varies from event to event (often information about the publishing job).

resource_id

The UUID of the target resource.

Example

Request POST data:

```
{
  "event_id": "pdf-ready",
  "event_data": {},
  "resource_id": "d2788650-b8cd-11e7-a79f-42010a800002"
}
```

Expected response:



HTTP/1.1 200 OK

① NOTE: The POST request from Heretto CCMS to the target system includes JSON data.



Webhook Status Request

The status request is sent from Heretto CCMS to check on the publish job that sends a webhook notification.

Overview

Type	Value
HTTP Method	GET
URI	Specified in the webhook configuration
Returns	XML describing the status of the webhook event

Request Headers

Type	Value	Description
Authorization	Bearer <code>secret-key</code>	

Response Headers

Type	Value
Status	<code>200 OK</code> if successfully received



Request Parameters

np_event_id

This is a UUID that is the unique identifier for this notification event.

Response structure

```
<status status="">
  <progress-message> </progress-message>
  <output-link> </output-link>
  <log-output> </log-output>
</status>
```

Property	Allowed values	Type
status	complete:success, complete:failed, in-progress	string
progress-message	Any string	string
output-link	Valid URL	ULR
log-output	Any string	string

Response Example

```
HTTP/1.1 200 OK
Content-Type: application/xml; charset=UTF-8
```



```
<status status="complete:success" >
  <progress-message>Publish completed successfully!</progress-message>
  <output-link>{output-link}</output-link>
  <log-output>
    300 pages created.
    No errors or warnings.
  </log-output>
</status>
```



Configure Webhook Timeout

When Heretto CCMS sends a webhook notification, it waits for 3 minutes to get a response. If it receives no response, it reports an error with the target server. You can configure the time that Heretto CCMS waits before it reports an error with the target server.

Before you begin

Ensure that:

- You are a Heretto CCMS administrator
- You have access to the target system
- You read through [Webhook Authentication and Security](#) on page 74

Procedure

- 1 In Oxygen, navigate to `/db/organizations/{org-name}/configuration/webhooks/config.xml`.
- 2 Add time in milliseconds like this: `<timeout>{timeout-in-milliseconds}</timeout>`

Example: To set the timeout to 5 minutes, add `<timeout>300000</timeout>`.

```
<webhook_config id="webhook_name">
  <name>My Webhook</name>
  <description>This is an example webhook</description>
  <notify_url>https://target.system.com/listening/uri</notify_url>
  <!-- optional -->
  <timeout>300000</timeout>
</webhook_config>
```

Next steps

Test your configuration by doing any of the following:



- [Send a Webhook Notification on Publish](#) on page 88
- [Send a Webhook Notification on Demand](#) on page 91



Send a Webhook Notification on Publish

Heretto CCMS enables you to automatically send webhook notifications on successful or any publishes.

Before you begin

Configure a webhook integration. See [Implement a Webhook Integration](#) on page 73.

Procedure

- 1 In the Content Library, click a map and in the right pane select **Output→Publish**.
- 2 From the list on the left, select a publishing scenario that enables you to send webhook notifications.
The scenario should have the following options available.

Options

Webhooks

Do not send

Debug publish

Do not send

Send on success

Send regardless of build result

- 3 Optional:
In the **Enter description** field, add a meaningful description of the publishing job.
Once you publish the document, you can see that description in the list of completed publishing jobs.
- 4 From the **Webhooks** drop-down menu, do any of the following:
 - To send a webhook notification on a successful publish, select **Send on success**.
 - To send a webhook notification on any publish (including failed publishes), select **Send regardless of build result**.
- 5 Set the publishing parameters.



Publishing scenarios are configured by your CCMS Administrator. The publishing scenario can include the following parameters.

Template

Enables you to select output style.

DITAVAL

Optional parameter. Enables you to select a DITAVAL file for conditional processing. This parameter may be pre-set and fixed in a publishing scenario.

Output type

Required parameter. Enables you to select the output format, for example, PDF or HTML. This parameter may be set to multiple options. In that case, the publishing scenario defaults to the first option on the list. This parameter may also be pre-set and fixed in a publishing scenario.

Locales

Optional parameter. Enables you to select a locale (target language) when publishing localized content. If your content is not localized, this parameter defaults to the source language.



Track changes

Optional parameter. Controls whether suggestions added in the Suggesting mode in the Content Editor are included in the PDF output.

- **Publish without track changes included (default)** - Generates output that appears as if all suggestions were rejected. Available in PDF Generator and DITA Open Toolkit.
- **Publish with track changes included** - Generates output that appears as if all suggestions were accepted. Available in PDF Generator and DITA Open Toolkit.
- **Publish with suggestions included** - Generates output with suggestions highlighted as they appear in the Content Editor, where added content is color-coded, and deleted content is color-coded and crossed out. Available in PDF Generator only.

☆ **IMPORTANT:** If your PDF Generator scenarios are not based on the default Color or Grey templates, include this import in your scenarios to ensure suggestions are highlighted:

```
@import url("https://jorsek.github.io/
pdfgen.github.io/styles1.1/modules/
styling/suggestions.css");
```

Users with the Administrator role in Heretto CCMS can edit publishing scenarios.

Webhooks

Optional parameter. Enables you to send webhooks on successful publish or any publish.

Debug publish

Optional parameter. Generates additional files that are added in the **debug** folder to the output ZIP file. You can use those files to troubleshoot publishing issues in your publication. This option is only available in DITA-OT and PDF Generator publishing scenarios for those users with Administrator privileges.

For more information about the DITA Open Toolkit (DITA-OT) parameters, see https://www.dita-ot.org/dev/parameters/parameters_intro.html.

6 Click Publish.

Result: The resource publishes. It may take a moment to complete.



Send a Webhook Notification on Demand

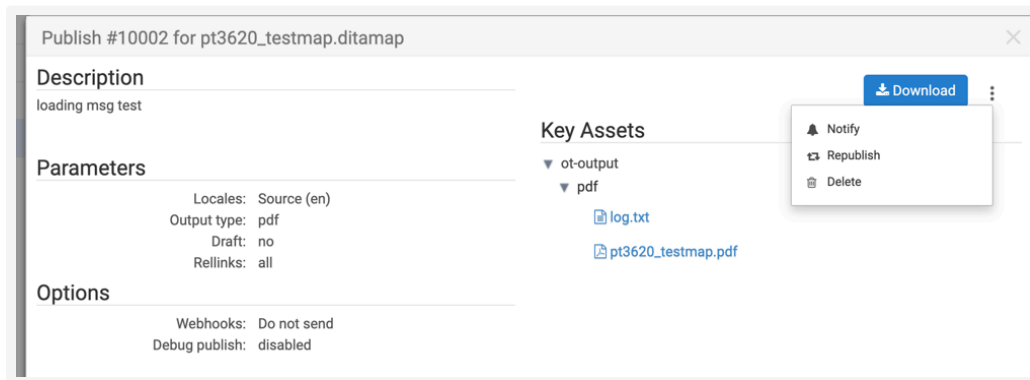
Heretto CCMS enables you to manually send webhook notifications from the **publishing** interface or from the Content Library.

Before you begin

Configure a webhook integration. See [Implement a Webhook Integration](#) on page 73.

Procedure

- To send a webhook notification from the **publishing** interface:
 - a In the Content Library, hover over a resource, and click the **Output** icon.
 - b In the publishes list, click a publish.
 - c In the Publish window, in the top-right corner, click the **more** button and select **Notify**.



Result: At the top, the **Notifications send** information appears.

- To send a webhook notification from the Content Library, right click a resource and select **Webhook Notification**.

① **NOTE:** This guide assumes that the **Webhook Notification** option is configured. Your webhook notification option might be implemented with a different name.